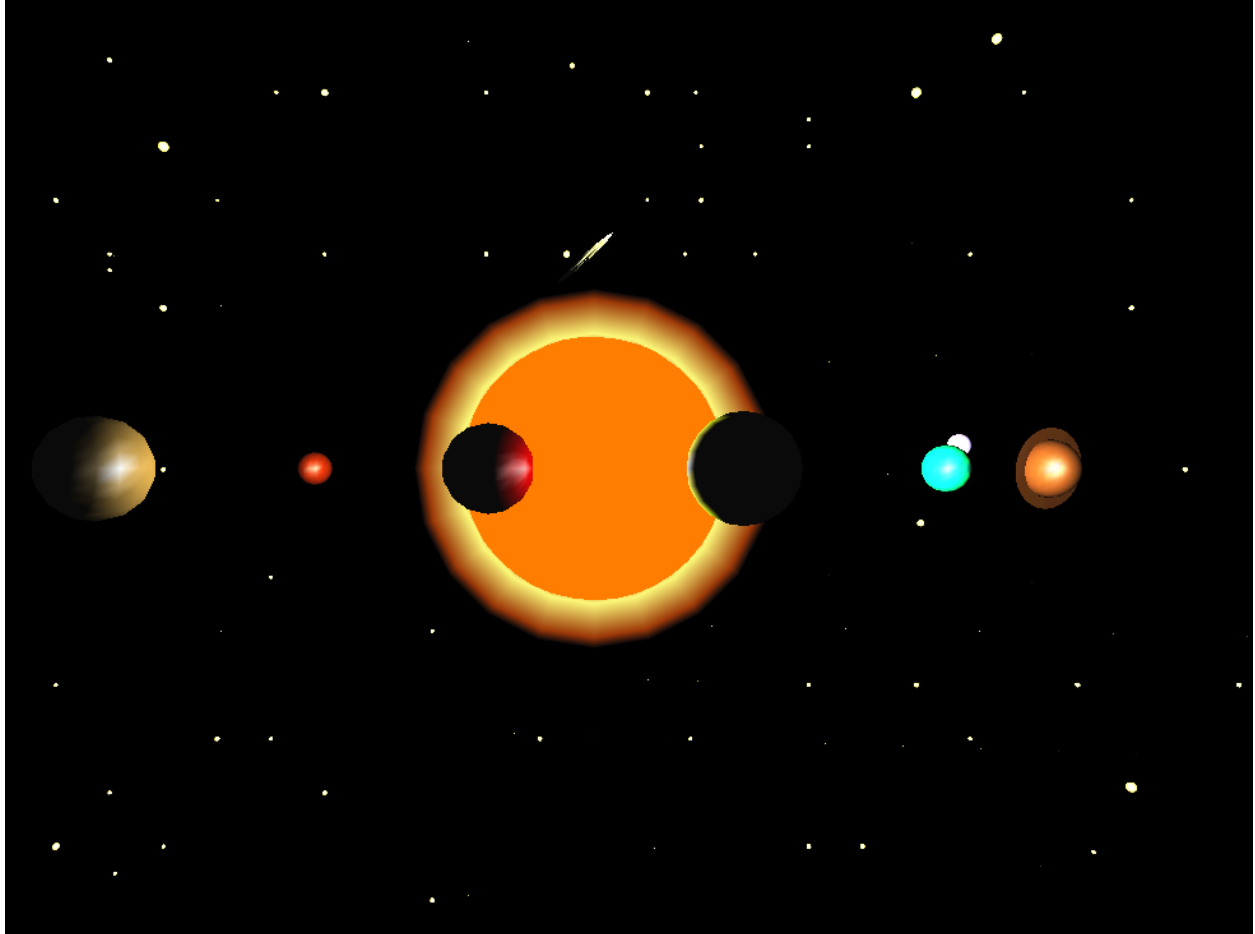


COMP2800: TEAM 4 PROJECT

Cosmic Explorer: A Journey Through Space and Beyond



04/04/2025

Executive Summary

This project developed an engaging Space Explorer game using Java3D and LWJGL, along with Blender for 3D rendering. The game starts off with the user in the Delta spacecraft, in front of them they see the Solar System, complete with all the planets orbiting around the sun. Each planet has a unique appearance that may not be accurate to the true image, but still provides enough context for the user to start their journey. There is an announcer that gives some details about the spacecraft for more information. The user can explore the solar system using their mouse, the sliders on screen, or even the keyboard. While exploring space, the black hole comes into view, and as soon as it does all the other planets and the sun go dark. Now the user must find the black hole to reach their destination. Once the user finds the black hole, a light tunnel warp sequence commences and the user is transported to a Farm created using LWJGL. The light tunnel sequence gives the appearance of travelling at light speed, with an eerie sound that builds anticipation. The Farm has a terrain with trees and a path, and the user is able to freely explore after arriving at their destination. The planning and execution of the project development was done within the Scrum Framework, with each feature being organized into backlog items. Then the backlog items were spread across “Sprints,” and then divided into manageable tasks which were completed by the team members. Overall, this project demonstrated the agile software development of a game using Object-Oriented Programming, Java Libraries, and external applications like Blender.

Statement of Honesty:

In submitting this team project and report, we confirm that our conduct during this project adheres to the Student Code of Conduct. We confirm that we did NOT act in such a way that would constitute cheating, misrepresentation, or unfairness, including but not limited to, using unauthorized aids and assistance, personating other persons, and committing plagiarism.

Signatures of all members:



Section 1 - Introduction

1.1: Purpose of the Project

The primary goal of the project was to develop an immersive 3D experience in Java that allows users to assume the role of a cosmonaut aboard the Delta spacecraft. This interactive simulation was designed to take users on a journey through a meticulously rendered solar system, where they navigate through celestial bodies, overcome alien threats, and ultimately transition into a uniquely stylized 3D farm world. The project aimed to merge technical skills with creative storytelling by combining advanced Java 3D rendering techniques, modern OpenGL features via LWJGL, and visually rich assets created in Blender. Ultimately, the project sought to demonstrate a robust, scalable engine capable of managing complex scenes and transitions while delivering a compelling narrative experience.

1.2: Issues To Be Discussed and Their Significance

Several key issues were addressed during the development of the project, each carrying significant implications for the overall success and user experience. One major challenge was integrating disparate rendering technologies—Java3D for the solar system simulation and LWJGL for the farm world—while ensuring smooth transitions between these vastly different environments. Another critical issue involved optimizing performance to maintain high frame rates and visual fidelity, even as the scene complexity increased with dynamic lighting, textured models, and real-time interactions. Additionally, the project tackled user interactivity challenges by implementing an intuitive camera system and input controls that allowed users to seamlessly navigate the virtual world. Addressing these issues was vital to creating an engaging and immersive experience that balanced technical complexity with narrative clarity.

1.3: Project Methods

The project was implemented using a combination of Java, LWJGL, and Java3D, leveraging object-oriented design to achieve modularity and maintainability. Agile methodologies, specifically the Scrum framework, were employed to manage development through iterative sprints and continuous feedback. The system architecture was divided into distinct packages—such as engineTester, entities, models, renderEngine, and textures—each responsible for specific aspects of the engine, ranging from display

management and model loading to real-time rendering and texture mapping. Advanced rendering techniques were applied throughout the project, including the use of Vertex Array Objects (VAOs), shader programming, and hierarchical scene graph structures to handle complex transformations and animations. Additionally, assets created in Blender were integrated into the codebase to enhance the visual quality of key scenes, such as the cockpit and the immersive transition tunnel that leads to the farm world. These methods combined to form a robust framework capable of delivering a dynamic, interactive 3D experience.

1.4: Limitations and assumptions

Despite its successes, the project faced several limitations and was built upon a number of assumptions. Performance constraints necessitated optimizations that sometimes compromised the level of visual detail achievable in real-time, particularly when managing resource-intensive scenes. The integration of different rendering libraries (Java3D and LWJGL) assumed a consistent runtime environment and sufficient system capabilities, which may not hold true across all hardware configurations. Additionally, the project assumed that users would have standard input devices and modern graphics hardware, potentially limiting accessibility for those with older systems. The transition between the solar system and the farm world, while innovative, required certain compromises in animation smoothness and asset detailing due to the inherent complexity of seamlessly integrating such disparate environments. Future iterations of the project could focus on further optimization, enhanced cross-platform compatibility, and improved user interactivity to address these limitations.

Section 2 - Project Management with Scrum

2.1: Use Cases as Backlog Items

One of the main components of Agile Software Development using the Scrum Framework is the product backlog. This backlog consists of several features which are formatted such a way that they are from the perspective of the user. Backlog items helped keep track of the overall features that the product required, and were updated as needed. Some backlog items had to be shifted between sprints because they took longer than expected to complete and this also helped organize the timeline of the project.

The Cosmos Exploration project had 12 backlog items in total that were completed by the deadline. The creation of these items was done with the agreement of all team members, and everyone was encouraged to provide feedback. There were a few backlog items that could not be done due to the time constraint but all measures were taken to ensure that the project still achieved the main idea.

The items were given titles in the format of user stories: “As a user, I want [feature] so that [benefit]”. Having the title be from the perspective of the user helped find the purpose for each feature. This brainstorming and thought process led to the creation of a more meaningful project where each step was taken for the benefit of the user.

Backlog items were also assigned effort in hours, and priority. Some of the items had more effort assigned than others, and due to this they were also of higher priority. The organization of the backlog items was done so that the higher priority ones were completed earlier on. The backlog items/user stories were also assigned one person that would take on the responsibility of the item. This did not mean, however, that no one else from the team could help in its completion. An example of one of the backlog items is one with the title “BlackHole portal: As a user I want a blackhole portal to begin my travel through universes so the transition is smooth.” This item was assigned to Kulsum Khan, the effort was set to 9, the priority to 2 (1 would be the highest priority). This item was originally in Sprint 1 —more explanation on the sprints to follow— but was shifted to Sprint 2 because it took longer than expected.

Since each backlog item is relatively general, they were divided into multiple tasks for a more structured approach to finishing them.

2.2: Creation of Tasks

The creation of tasks was more dynamic and flexible compared to the creation of the backlog items. Tasks were added and deleted as needed, and it was not necessary that all the tasks for the backlog item were assigned to the person that was assigned the item. If a team member finished their task early or did not have any current backlog items to work on, they were assigned tasks for other backlog items, to ensure full commitment and contribution to the project.

The scope of the task was specific enough so there would not be confusion about what was expected, but it was not so narrow that no initiative could be taken by the team member working on it. When the backlog item was created it was given some basic tasks,

and as the progress went on, more tasks were added, deleted, or edited.

Building upon the previous example, there were 5 tasks for that backlog item. In this case all tasks were assigned to Kulsum Khan, but in Appendix A the backlog items with tasks assigned to different people are mentioned. Each task has a “Remaining Work” section where an estimate of the hours remaining till completion is added. The tasks are automatically given the same priority as the backlog item.

There was more freedom with the tasks, as the person assigned the backlog item was allowed to create tasks themselves as needed. This helped them make a plan that they understood and could follow.

2.3: Planning and Carrying out the Sprint

The team had an initial meeting on February 26th to start our first sprint. Since it was the first time, the meeting was a bit longer so everyone could understand what was happening. We assigned some roles, and brainstormed some backlog items, and which sprint we would place them in.

We had 2 sprints, and each were around the same length, the first one ranged from February 26th to March 19th, and we covered 4 backlog items in this sprint. The second sprint was from March 20th to March 30th (the 30th was the due date of the video), and we covered the remaining 9 backlog items in the second sprint. The example backlog item was originally in Sprint 1 but was shifted to Sprint 2 because it took longer than expected.

The official daily scrum meetings took place every Monday and Wednesday at 2:30pm EST, during lecture hours as everyone would be free anyway. In the beginning of the project 4/5 of the group members met in person, with Hamnah joining in a Microsoft Teams meeting online. Towards the middle though as people got busy with other things we shifted to fully online meetings that were recorded for future reference. If anyone was unable to attend a meeting and we could not reschedule, we kept them updated through the Microsoft Teams group chat and Azure boards. We also had multiple mini meetings during the project, with subsets of the group members for discussing specific issues. The whole team was briefed about the contents of these meetings during the Daily Scrums.

Following the sprint planning as best as possible helped us complete majority of our backlog items, even when faced with some difficulty due to some items or tasks taking

longer than expected.

2.4: Team Management

In the Scrum Framework, each team member takes on a role. In our team, Kulsum Khan was chosen as Scrum Master, meaning she would have the responsibility of planning meetings, checking everyone's progress and managing the Azure board. Makhsoma Khamzaliyeva was chosen as Product Owner as she had the most initial ideas for the direction and tone of the project. All of the members took on other roles as well like Developers and Testers. Assigning roles helped us follow the framework better, and maintain consistency throughout the project. The members were expected to fulfill the role they were given and take full responsibility.

Progress was also checked daily by the Scrum Master to ensure the team was on track to finish in time. The team members also gave updates themselves in the Microsoft Teams group so everyone was coordinated. If there were any issues with the backlog items they were shared and other team members tried their best to help resolve them. If the issues were regarding the member the team discussed and proposed solutions with as minimal involvement from the Professor as possible.

Section 3 - Object Oriented Software Development

3.1 Identification of Classes

The solar system simulation was implemented using several Java3D that formed the foundation of the program's architecture. The main class, *SSEngine*, extended *JFrame* to create the application window and manage the user interface components. This class housed the *Canvas3D* viewport which served as the rendering surface for all 3D graphics. The *SimpleUniverse* class acted as the root container for the entire 3D environment, handling essential viewing platform configuration and camera management through its *ViewPlatformTransform*. *TransformGroup* nodes played a critical role in managing hierarchical transformations, enabling three distinct types of planetary motion: orbital

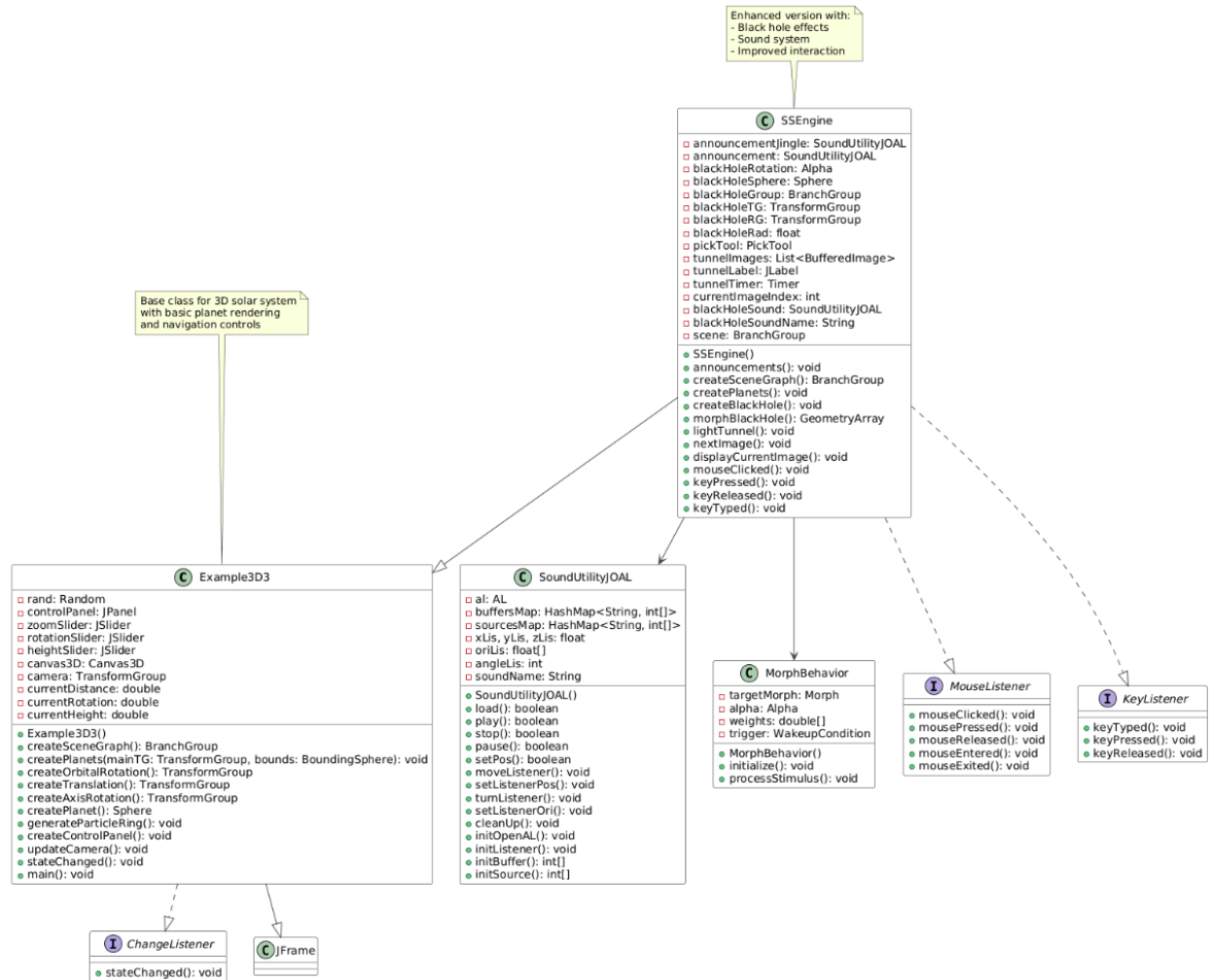
rotation around the sun (created via *createOribitalRotation()*), axial rotation (*createAxisRotation()*), and positional offsets (*createTranslation()*). These nodes were also used for the translation and rotation of the BlackHole. For visual rendering, the *Appearance* class combined texture maps with *Material* properties to give each planet its distinctive look, while the *TextureLoader* class handled the loading and mapping of planetary surface images from JPG files to spherical geometries. The *Appearance* class was also used on the BlackHole for an opaque and dull look. The use of *PickTool* objects helped enable picking of the BlackHole which allowed for the light tunnel sequence. The *SSEngine* class also implemented *MouseListener*, *ChangeListener* and *KeyListener*, all of these classes playing a vital role in navigating through space. The *MouseListener* (*mouseClicked()*) allowed the user to move around using their mouse, and also click on things. The *KeyListener* (*keyPressed()*) further improved navigation by providing a way to move around using the keyboard. The *ChangeListener* was used with JSliders to change the camera view.

In addition to the solar system simulation, the farm scene was implemented using LWJGL, forming the core of the game engine that drives the transition into a 3D farm world. This part of the project is organized within several dedicated packages, each encapsulating key functionalities to ensure a modular and maintainable design. The *engineTester* package, for instance, contains the *MainGameLoop* class, which initializes the display via the *DisplayManager*, loads 3D models and textures using the *Loader* and *OBJLoader* classes, and orchestrates the rendering process through the *MasterRenderer*. The game engine leverages modern OpenGL techniques, utilizing Vertex Array Objects (VAOs) and shader programs to efficiently render complex scenes. Entities such as trees, farm structures, and other assets are instantiated as *TexturedModel* objects and positioned throughout the terrain using randomly generated coordinates, creating a dynamic and interactive environment.

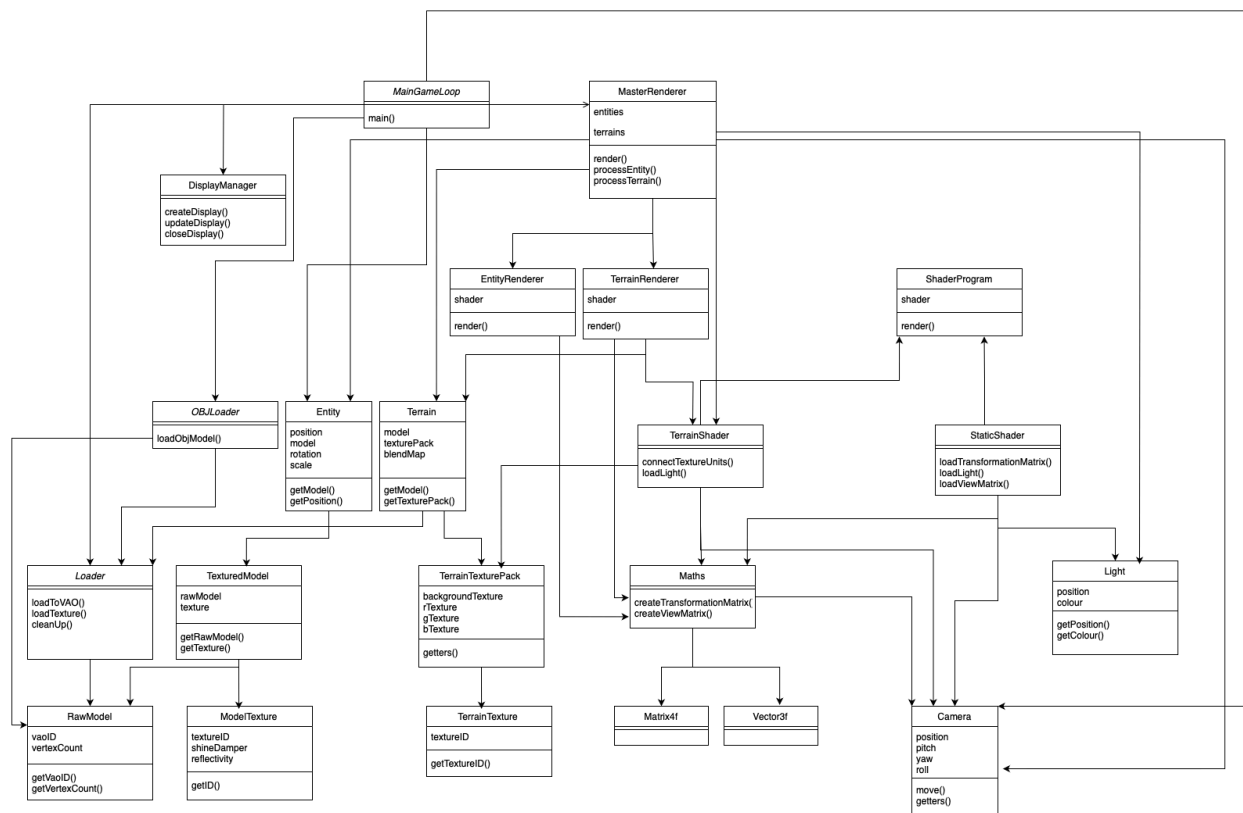
The farm scene's navigation and camera management are handled by a dedicated *Camera* class, which processes user input via keyboard to offer smooth, real-time movement through the environment. This system is tightly integrated with the overall game loop, ensuring that scene updates, rendering, and user interactions are synchronized seamlessly. The combination of these LWJGL components not only facilitates high-performance rendering but also allows for future expansion, such as the addition of more complex lighting models, interactive gameplay elements, and further enhancements to the farm world. Together, the Java3D solar system simulation and the LWJGL-based farm scene demonstrate the project's dual approach to creating immersive 3D environments, each leveraging specialized tools and techniques tailored to its unique artistic and technical challenges.

3.2: Software Design with Class Diagram(s)

The program's architecture followed a structure scene graph design that efficiently organized all 3D elements. At the top level, the *SSEngine* class contained the *SimpleUniverse* instance, which managed the complete viewing environment. This design employed composition, with *SSEngine* owning and controlling the *SimpleUniverse*. The *TransformGroup* nodes formed the backbone of the planetary motion system, using aggregation to contain and manage individual *Sphere* objects representing planets. These relationships created a parent-child hierarchy that allowed complex celestial mechanics while maintaining clean code organization. The *Appearance* class depended on *TextureLoader* for image data, demonstrating a clear dependency relationship. This hierarchical structure proved particularly effective for managing multiple moving parts, as each planet could be transformed independently while maintaining its position in the overall solar system. The design's modularity made it straightforward to add or modify planetary objects without affecting the entire system.



As for the Farm scene there were many classes involved as LWJGL was used for its creation. There were many classes used for each aspect of the program. For example there were *Camera* and *Light* objects used, as well as different Renderers that were used for the terrain and trees. Due to the farm scene being connected with many other parts of the LWJGL environment, the program also benefited from flexibility and modularity.



3.3: Techniques of Implementation

For the whole project many imports were required. Majority of them were from javax.media, some were from com.sun.utils and others from org.jogamp. The coordination of these imports was extremely important in getting the program to work. There were also several .jar files that had to be added to the project such as vecmath, com.sun.utils etc.

3.3.1: Solar System

The implementation leveraged several advanced Java3D techniques to create a realistic solar system simulation. The hierarchical scene graph approach allowed for efficient management of planetary motions, with each planet being a child of its own *TransformGroup* node. This structure enabled the program to handle complex orbital mechanics through relatively simple code, such as the *createOrbitalRotation()* method that established each planet's revolution around the sun. Texture mapping played a vital role in the simulation's visual fidelity, with the *TextureLoader* class mapping high-quality surface images onto spherical geometries. For Saturn's rings, the program employed a

procedural generation technique that created 700 individual particles in randomized positions, using mathematical calculations to ensure proper ring formation. Performance optimization was achieved through several methods, including pre-compilation of the scene graph via *BranchGroup.compile()* and careful reuse of *Appearance* objects for similar materials. The camera system, controlled through sliders in the GUI as well as keyboard commands, utilized *Transform3D* operations to provide smooth navigation through the solar system.

3.3.2: BlackHole

Introduction

The Black Hole was created using Java3D and Blender 4.3.2. The combination of the two helped create an animated and dynamic Black Hole travel experience for the user. For the black hole itself, it was created using a Sphere primitive shape from Java3D. It was given a custom appearance and material to ensure opaqueness and no shine.

Morphing of BlackHole

The main techniques that went into creation of the black hole were PickTools and Morph. There was an attempt to use the same *MorphBehavior* object as was used in a previous lab assignment, however there were issues because the imports were different. To combat this, a *MorphBehavior* class was found online (Froese, n.d.) which was used in place of the import. The morphing of the black hole was done by manipulating the *TriangleStripArray* of the original sphere. The *morphBlackHole()* method was called to create manipulated geometries for the Morph object. This method took the original geometry of the black hole, counted the vertices, retrieved coordinates and scaled them based on arguments. Then a new *TriangleStripArray* geometry was made which was assigned the scaled coordinates and returned back to *createBlackHole()*. A total of 3 scaled geometries were made that were placed in the Morph object. The morph object was added to *blackHoleRG* (rotation transform group), which was then added to the *blackHoleTG* (translation group). The *MorphBehavior* object, and *RotationInterpolator* for the blackHole were also added to the *blackHoleTG*. Finally, *blackHoleTG* was added to the black hole branch group.

The Morph object was given *userData* so it could be recognized when the mouse was clicked onto it. There was a *PickTool* created for the black hole branch group, and it was set to choose by geometry. The *mouseClicked()* method—which is implemented from

MouseListener— was used along with the PickTool. In this method, the point of clicking is found and a shape ray is set to the point, then the pickTool picks the closest object. After that, it recognizes the object chosen and if the user data is 3 the lightTunnel() method begins.

LightTunnel

As for the light tunnel, this was created in Blender 4.3.2 using Cylinders and Spheres. The tunnel consists of many duplicated cylinders, and the sphere is used to control the hairs on the cylinder. The emission on the sphere is set high, and that causes the hairs on the cylinder to light up. As the user approaches the end of the tunnel the emission becomes white and even brighter so the transition from tunnel to farm is

smoother. To create an animation effect of traveling through the tunnel, a camera was placed at the beginning of the tunnel, and then the animation slider was dragged to the end. At the end, the camera location was moved to be at the end of the tunnel. This way, when “play” was pressed in Blender, the camera moved through the tunnel. The camera view of this animation was then downloaded as a collection of PNGs. The PNGs were placed into a BufferedImages List and were looped through in the *lightTunnel()* method. As the images were displaying; a timer was used for controlling the loop, because it had to sync with the sound of the black hole.

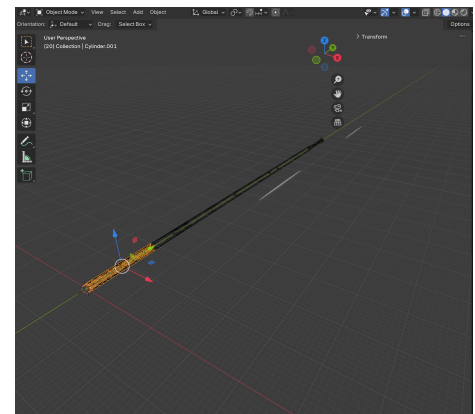


Figure 3.3.A. Line of mesh cylinders used to create tunnel (the “top” and “bottom” faces were removed)

BlackHole Sound

This was another technique used for the BlackHole: SoundUtilityJOAL. This class was borrowed from a previous assignment (Yuan, n.d.), but some changes were made to it to allow it to be used to play multiple sounds. In this case the blackHoleTransition.wav file was played as the user traveled through the tunnel. Once the images ended, the timer

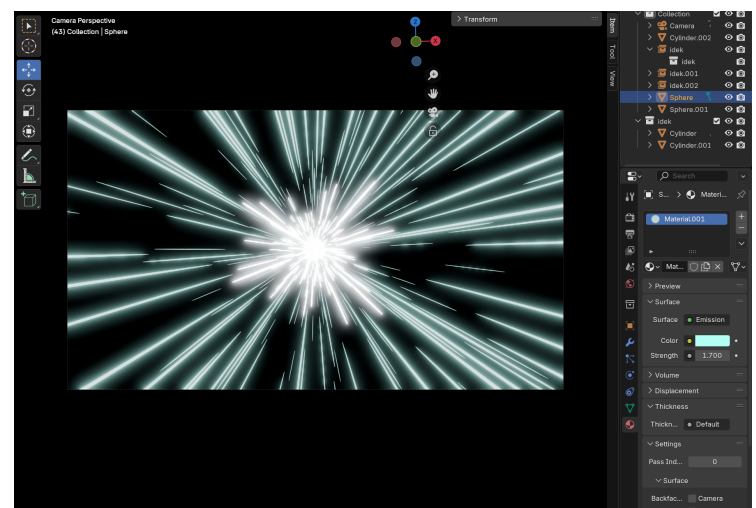


Figure 3.3.B. BlackHole tunnel in Blender, displaying the emission using a Sphere

stopped and the sound stopped with it.

All of these techniques combined were used in the creation of the BlackHole element of this project.

3.3.3: Blender SpaceCraft Rendering and Scene Design/Conceptualization

Introduction:

The 3D scene was designed to immerse users in a dynamic environment that begins in a scientifically accurate solar system and transitions into an imaginative, quasi-planar farm world upon entering a black tunnel. This shift in worlds required distinct artistic and technical approaches for each space. The cockpit was the first major environment developed, intended to provide a realistic and immersive interface from which the solar system could be explored.

Rendering The Cockpit:

The design process began with a conceptual blueprint that mapped out the spatial relationships between key components, such as the steering wheel, buttons, screen, and structural frame. Primitive modelling techniques were used in Blender to shape the cockpit's foundational elements. Cylinders were used for the steering wheel and vent components, while cubes were modified through scaling, extrusion, and beveling to create buttons, the dashboard, and structural supports. Special attention was given to material realism—such as applying partial transparency to glass and using metal-like shaders on support brackets—to enhance visual fidelity.

Texture mapping played a crucial role in refining the cockpit's realism. Procedural textures and image-based textures were applied to various surfaces to enhance their material properties. The dashboard and structural panels were assigned a brushed metal texture using UV mapping, ensuring proper alignment and realistic reflections under lighting conditions. The glass canopy was given a semi-transparent shader with adjusted roughness and reflection values to create the effect of light refraction. Additionally, normal maps were incorporated to add fine surface details—such as grooves on components—without increasing the polygon count.

To maintain hierarchical organization and allow smooth animation, all individual elements were parented to the wheel object, enabling cohesive transformations such as rotation and translation. This rigging setup was essential for future interactive features. During this phase, several issues arose, particularly around precision alignment.

Components like the buttons and dashboard were initially misaligned due to inconsistent origin points. This was resolved by resetting object origins and using Blender's snapping tools to align objects accurately. Another issue involved lighting—the initial renders had harsh shadows and uneven highlights. We corrected this by refining the HDR lighting setup, repositioning light sources, and fine-tuning shadow softness settings in the Render Properties panel.

Additionally, a major challenge encountered during texture mapping was ensuring that high-resolution textures were mapped accurately without distortion. Initially, some materials stretched due to improper UV unwrapping, particularly on curved surfaces like the vent structures. This issue was resolved by manually adjusting UV seams and using Blender's UV projection methods, such as cylindrical mapping, to achieve a natural distribution.

Photos:

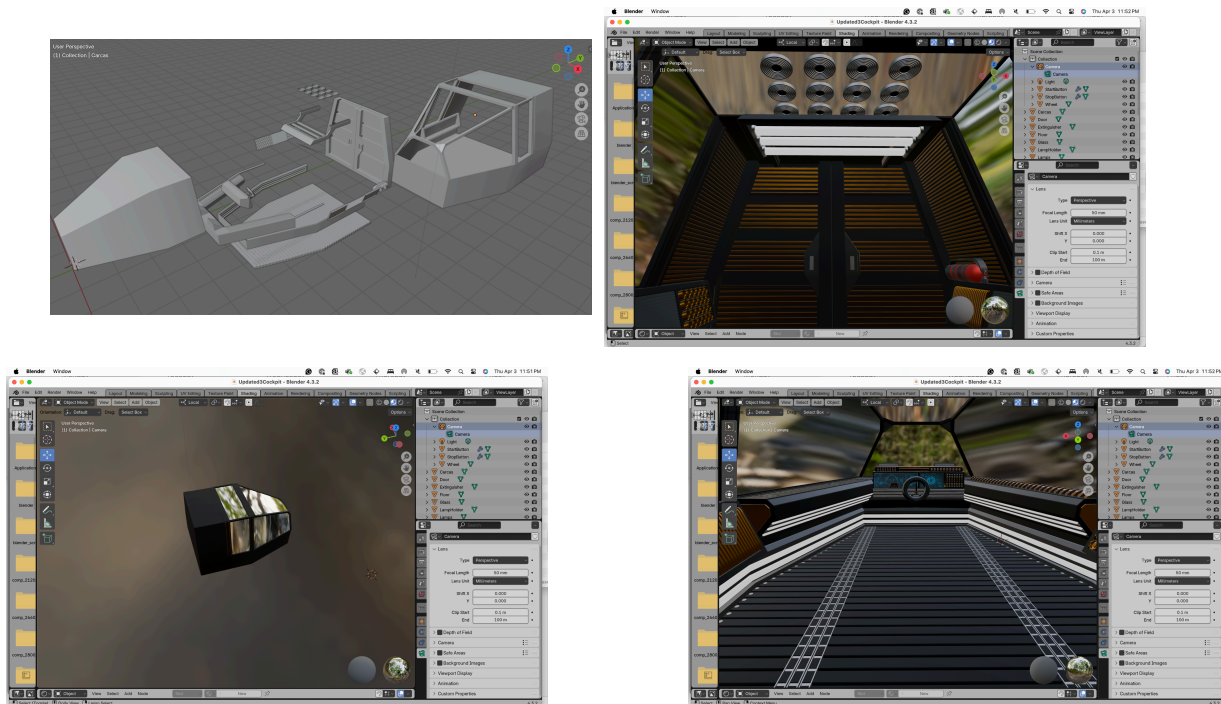


Figure 3.3.C. Screenshots displaying the cockpit rendering in Blender

Once the spacecraft rendering was complete it was imported into the program as a PNG. It stays with the user as they travel through space and the BlackHole. When the program

first runs, there is an announcer voice which is imported using SoundUtilityJOAL (Yuan, n.d.). The announcer gives some context to the user regarding the situation.

3.3.4: Spacecraft Movements and Navigation

The development of the spacecraft movement system began with the intention of fully replicating a Blender-based control mechanism within a Java3D environment. In Blender, the spacecraft was designed as a 3D object whose movement was controlled using Python scripts. These scripts enabled smooth forward and backward thrust through a lever and rotational steering via a wheel, all implemented using object transformations, interpolation, and damping. Our goal was to bring this same interactive and immersive experience into Java3D by translating the Python-based logic and animations into Java equivalents.

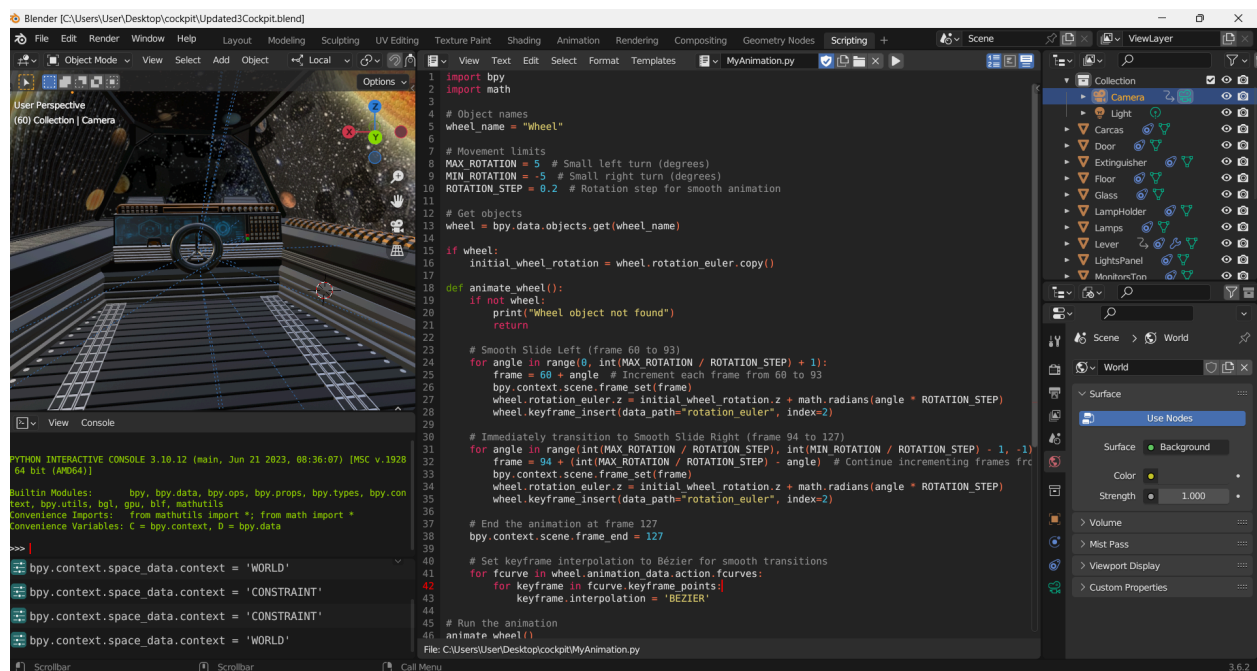


Figure 3.3.D. Screenshot displaying the cockpit and scripting for Wheel movement and Lever movement implementation in Blender python Simple UI

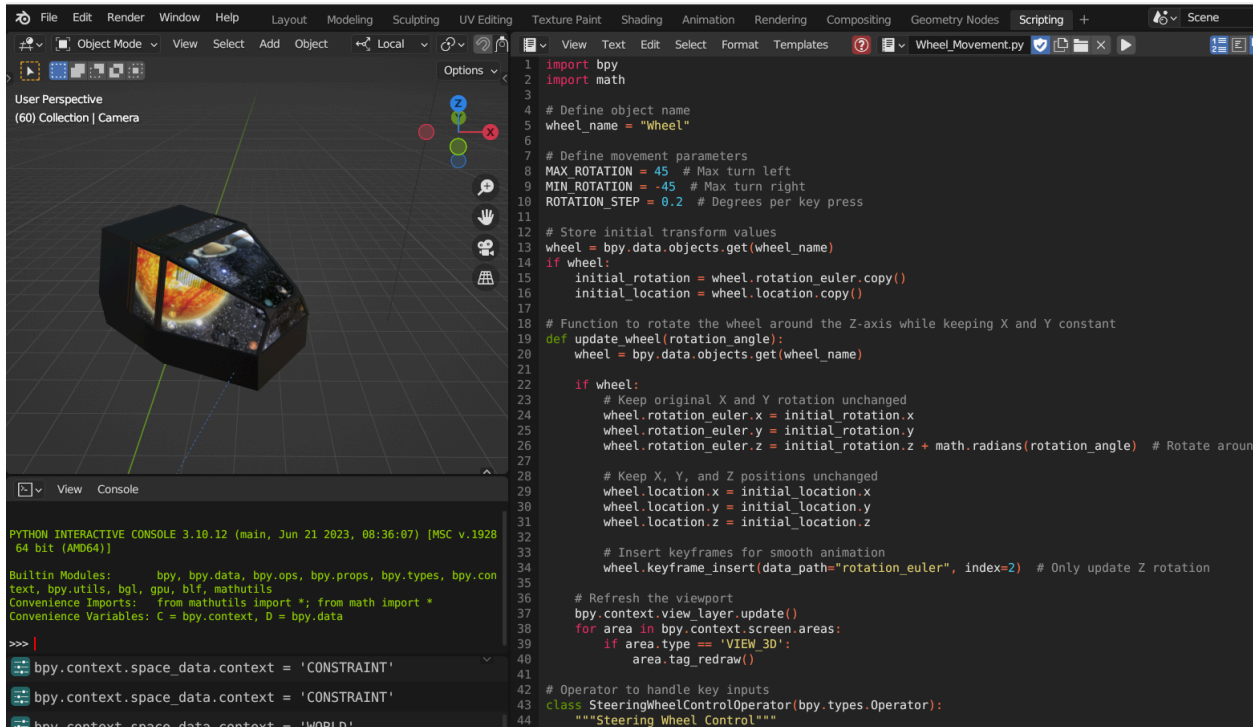


Figure 3.3.E. Screenshot displaying the Spacecraft moving along the Y-Axis upon using the steering wheel

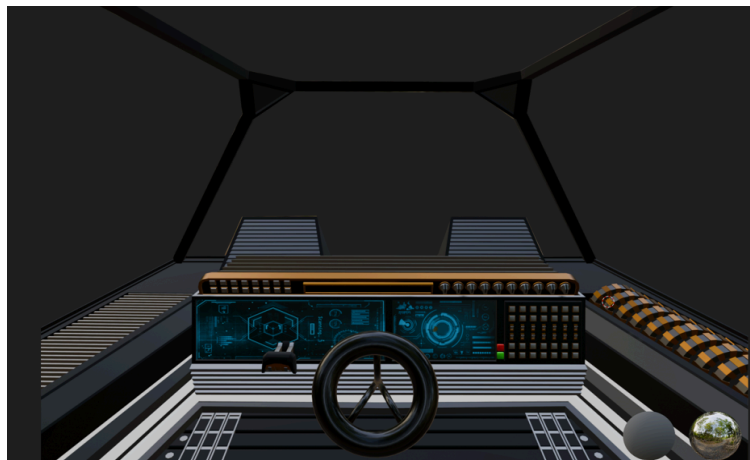
We initially attempted to import the entire spacecraft model from Blender into Java3D as a .obj (Wavefront) file. The plan was to include not only the model's visual elements but also replicate its interactive behaviors using Java3D's TransformGroup, Transform3D, and event-handling interfaces. The forward and backward movement, originally handled by manipulating the lever's position along the Z-axis with smooth interpolation in Blender, was re-implemented in Java3D by adjusting transformation values based on keypress events ('U' and 'D' keys). Similarly, steering was mimicked by rotating a wheel object left and right with key inputs ('L' and 'R'), again using smooth rotational transformations to maintain the natural motion present in the original Python version. This effort required importing key Java3D libraries such as vecmath.jar, j3dcore.jar, and j3dutils.jar, which provided the tools necessary for precise manipulation of 3D objects and scene graph management.

However, despite the detailed effort to convert the system, we ran into significant challenges with importing the Blender-exported .obj model into Java3D. The model

experienced multiple technical issues, including broken or missing textures, transformation errors, and the inability to preserve Blender’s animation and interaction logic. The .obj format lacked support for dynamic behaviors like damping and smooth real-time updates, which were integral to the Blender version’s realism. We tried several solutions, including tweaking material files and restructuring transformations manually within Java3D, but none provided a satisfactory or stable result.

As a result, we pivoted to a new, more practical solution: using a 2D cockpit image overlay (cockpit.png) to replicate the spacecraft interface. This image was edited to include transparent windows and was applied as a foreground texture in the Java3D scene, simulating a first-person view from inside the spacecraft. It functioned as a static overlay or HUD that always stayed in front of the user’s view, giving the impression of being inside the cockpit. To simulate forward and backward movement—originally controlled by the lever—we linked the cockpit to the zoom sliders already available in the solar system code. These sliders controlled the camera’s distance from the scene, and thus, moving them forward or backward created the illusion of thrust or reverse motion, very similar to the lever-based control from Blender.

Figure 3.3.F. Screenshot displaying the cockpit.png with transparent windows to provide first person overlay



Directional movement was recreated by mapping keypresses to rotate the camera left and right around the Y-axis. This provided a steering effect akin to the wheel control from the original Python implementation. Although this approach lacked a rotating 3D wheel object, the visual change in camera orientation relative to the cockpit overlay achieved the same interactive result.

Ultimately, settling on the image overlay approach allowed us to maintain the spirit and feel of the original Blender project while avoiding the complex and error-prone process

of importing and animating a 3D spacecraft model in Java3D. It provided a lightweight, intuitive, and visually consistent experience, capturing the essence of the original control system through thoughtful design and adaptation. This solution proved to be not only technically effective but also a creative reinterpretation of the project's original goals.

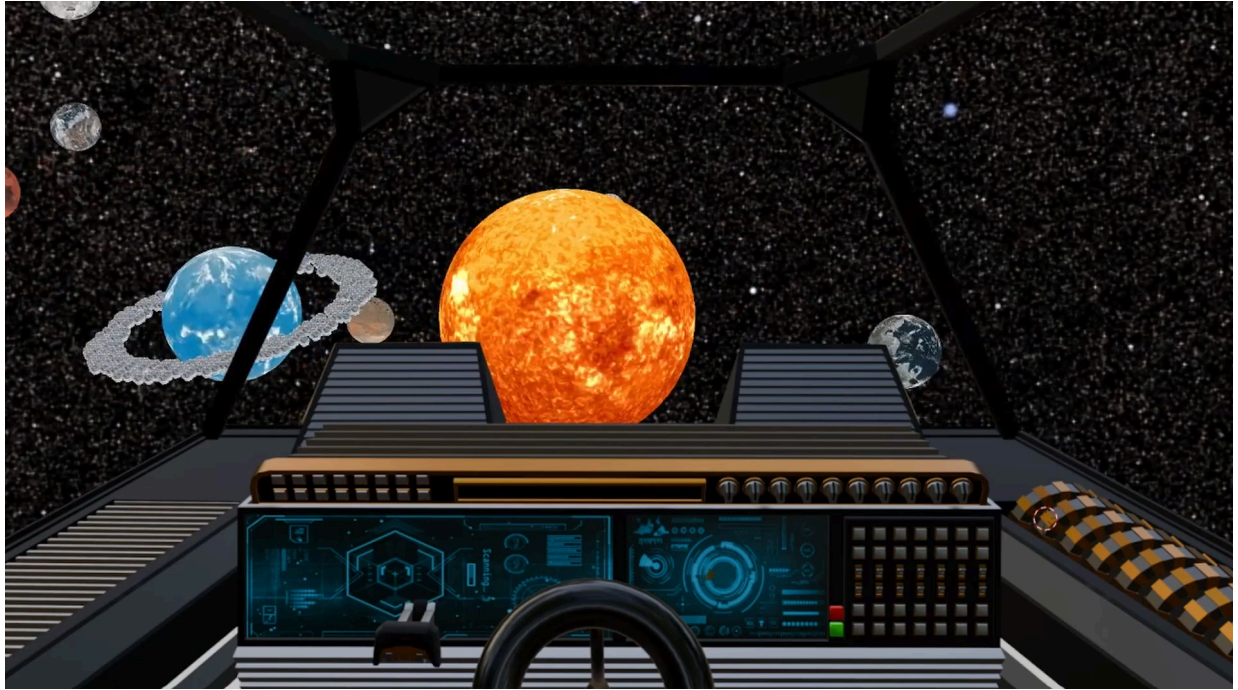


Figure 3.3.G. Screenshot displaying the new Spacecraft cockpit solution in the solar system with all the movement mechanisms

3.3.5: Farm Scene made with LWJGL

For the farm scene in the project, the implementation techniques were centered on creating a modular, maintainable, and high-performance 3D engine using Java and LWJGL. The design adopted an object-oriented architecture, separating the farm scene into distinct packages—each responsible for different aspects of the engine. For example, the `renderEngine` package handles everything related to display management, model loading, and rendering, while packages such as `entities` and `models` encapsulate the logic for in-game objects and their data structures. This separation of concerns not only improves code clarity but also simplifies debugging and future enhancements.

Use of OpenGL:

A key technique was the use of modern OpenGL features through LWJGL, enabling

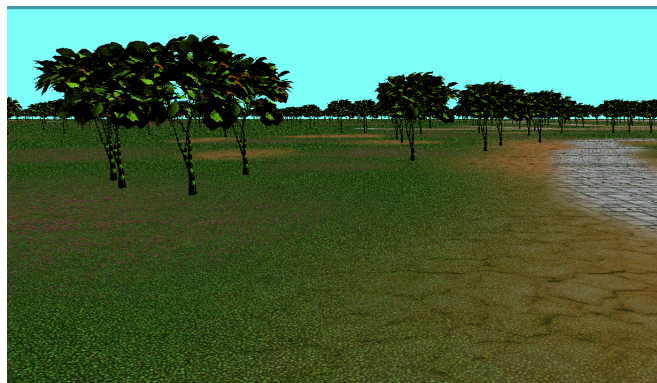
efficient rendering of complex 3D scenes. The engine loads 3D models using an OBJLoader that parses .obj files and converts them into optimized data structures (i.e., RawModel and TexturedModel), which are then uploaded to the GPU via Vertex Array Objects (VAOs). Textures are similarly managed, with a dedicated Loader class responsible for interfacing with OpenGL to load texture data. This approach ensures that graphical assets are rendered efficiently, with minimal overhead, while also supporting the dynamic aspects of the scene, such as lighting and shading.

Scene Navigation:

Camera management and scene navigation are achieved through a dedicated Camera class that processes real-time user input. By continuously updating the camera's position and orientation within the main game loop, the engine provides a smooth and interactive experience. The DisplayManager plays a crucial role in this setup by managing the lifecycle of the OpenGL window and ensuring a consistent frame rate through proper display updates and cleanup routines. This well-structured game loop not only drives the rendering process but also seamlessly integrates user input, thereby enhancing overall interactivity.

Overall, the implementation leverages modular design, efficient data handling, and real-time user interaction to construct a robust 3D engine. By combining well-defined packages, modern OpenGL techniques, and a responsive game loop, this part of the project demonstrates how complex scenes can be built incrementally. This foundation not only fulfills the immediate project goals but also sets the stage for future enhancements such as more advanced lighting, dynamic terrain modifications, and interactive gameplay elements.

Figure 3.3.H. Screenshot displaying the Farm terrain



3.3.5: Other Farm Assets

Rendering The Farm Assets:

Parallel to the cockpit, development of the animated farm world began with hand-drawn concept art created in Procreate. The characters—Moobaa, Dunny, and Qubee—were first rendered in 2D using expressive, stylized forms. These illustrations were imported as PNG files into Blender and converted into Grease Pencil strokes. From there, they were transformed into 3D polygonal meshes using the "Convert to Mesh" function. This conversion process required detailed vertex cleanup and extrusion to produce a quasi-planar 3D look that retained the charm of the original drawings. Artistic textures—sourced from the Procreate layers—were UV-mapped onto the final models, preserving the hand-drawn aesthetic while enabling them to function as textured 3D assets.

This same process was used to create environmental elements like the barn, hay bales, tractor, windmill, and fencing. Maintaining stylistic consistency across these assets was critical, so the workflow—draw, import, convert, clean, texture—was applied rigorously.

One technical challenge emerged during this stage: some Grease Pencil conversions resulted in erratic mesh topology, particularly with curved lines. These were corrected by manually merging vertices and simplifying geometry in Blender's Edit Mode. Another issue involved distorted textures when mapping the Procreate artwork onto extruded surfaces. This was resolved by adjusting UV seams and re-unwrapping the meshes with more strategically placed anchor points.

Although the animated farm world has not yet been fully integrated into the interactive scene, all assets are complete and optimized. The next step will involve importing the farm models into the 3D environment and transitioning to them via the black tunnel. This transition requires careful management of scale, lighting contrast, and camera animation to create a seamless and immersive shift between the hyper-realistic solar system and the stylized, planar farm.

Photos:



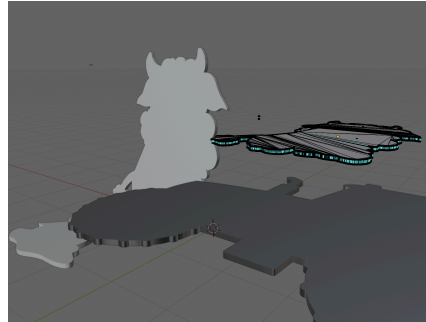
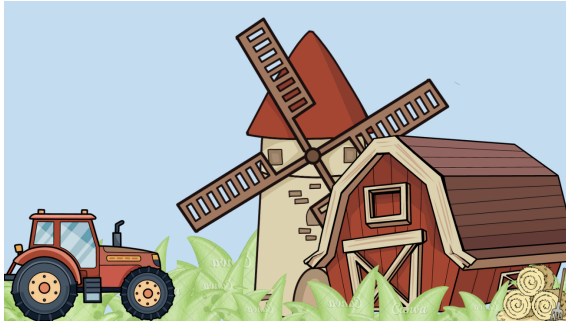


Figure 3.3.1. Collection of images displaying the Farm Animals, Windmill, and Tractor in Procreate and Blender.

3.4: Software Testing and Operation

The testing process for the Cosmos Exploration simulation involved multiple validation methods to ensure proper functionality and performance. Visual testing confirmed that all planetary textures loaded correctly and maintained their integrity during motion. Particular attention was paid to orbital mechanics, with extensive testing conducted to verify that planetary paths did not intersect unexpectedly. This required careful adjustment of orbital radii values in the orbitals array until all collisions were eliminated. The interactive components, particularly the zoom and rotation sliders, underwent stress testing to guarantee smooth camera movements under various operating conditions. Performance metrics, including frame rate analysis, were monitored during maximum scene complexity when all ten planets and Saturn's rings were visible. The program's operational workflow began with a careful initialization sequence that loaded all textures and built the scene graph before compilation. During runtime, the camera system updated through *Transform3D* operations in the *updateCamera()* method, while Java3D's internal scheduler handled animation loops. One significant debugging challenge involved resolving planet-ring collisions, which was addressed by properly scaling *BoundingSphere* sizes relative to orbital radii. These comprehensive testing procedures ensured the simulation's stability and visual coherence throughout user interactions. There was a feature that could not be implemented completely due to it not passing the tests smoothly. The project was supposed to have Aliens attacking the Delta Spacecraft at random intervals by throwing "goop" on it. However, there was an issue with the randomized timing as well as the

appearance of the goop the aliens were attacking the spacecraft with. Extensive testing of the feature revealed that it could not be a reliable addition to the project, and debugging it would take too long, and thus it was removed.

In parallel, the farm scene—implemented using LWJGL—underwent its own series of rigorous tests. The farm environment's assets, loaded via OBJLoader and managed by the Loader class, were evaluated to ensure that models and textures rendered accurately with no visual artifacts. Navigation within the farm scene was tested extensively; the dedicated Camera class was subjected to various user inputs (keyboard) to confirm that movement remained fluid and responsive under different operating conditions. Stress tests were performed on the MainGameLoop to assess the integration of LWJGL components, ensuring that the display updates managed by DisplayManager maintained a consistent frame rate even when the scene contained numerous dynamically loaded entities. Moreover, the transition effect from the solar system to the farm world—triggered by the light tunnel sequence—was validated for seamless execution, confirming that texture transitions, sound effects, and scene switching occurred without perceptible delays or performance degradation.

Together, these testing procedures not only verified the individual stability and performance of both the solar system and farm scene components but also ensured that the overall Cosmos Exploration experience delivered a coherent and immersive user experience.

Section 4 - Discussions and Recommendations

4.1: Discussions about Team Projects

Team projects are important for developing communication and coordination skills. Especially in fields like computer science where there are varying skill sets it is imperative to know how to work with other people. Within this group project there were some issues but they were resolved maturely within the group. At first there was a member who we could not reach, which was an inconvenience but once coming into contact with them we realized the issue and decided to work through it as a group. Furthermore there was an issue with a group member taking responsibility for their work and there were a few concerns regarding their work ethic. To resolve this issue a one-on-one meeting was held with a chosen spokesperson and the group member. The conversation helped clear out any confusion and misunderstandings and the group was

able to progress forward with only minor consequences. These issues however slightly held back the full potential of the group project. There were many things we wanted to implement but were unable to due to the nature of these issues. Resolving the problems became first priority which meant it was harder to focus on the software development aspect. Some of the minor features of the project could not be implemented, such as the alien goop effect.

4.2: Recommendations for Better Practices

Through this project we became more familiar with the Scrum Framework and also all the software development tools we used. Since this project was the first time the whole group was actually working with Java3D there were many setbacks we faced due to our lack of knowledge. After gaining this knowledge and becoming more familiar with things like Java3D, Blender and LWJGL, future projects will become easier. Also for next time it would be beneficial for the Scrum Framework to be set out better. There was some trouble with the backlog items and getting started with Azure DevOps, so it would be beneficial to brainstorm properly and get the Scrum process started as soon as possible.

Section 5 - Conclusions

To conclude, this project was extremely insightful regarding not only Java3D and other software tools, but also Agile Software Development and team work. It was a beneficial experience for us to learn about how realistic software development happens in the workforce and has helped us prepare better for our career. All team members gained valuable knowledge regarding Java3D development and how to coordinate projects with multiple team members, where the group is self-managed.

Section 6 - References

Froese, R. (n.d.). *MorphBehavior.java* (Version unknown) [Source code]. Retrieved from <https://personal.math.ubc.ca/~rfroese/java/reedtalk/MorphBehavior.java.html>

ThinMatrix. “OpenGL 3D Game Tutorials.” YouTube, 2014, www.youtube.com/watch?v=VS8wIS9hF8E&list=PLRIWtICgwaX0u7Rf9zkZhLoLuZVfUksDP&index=1&ab_channel=ThinMatrix

Yuan, Xiaobu (n.d.) SoundUtilityJOAL.java (Version unknown) [Source code]. Retrieved from Assignment 4 Files.

Appendix A: List of Backlog Items and Tasks With Brief Descriptions

Base Solar System - As a user I want a 3D Solar System environment so the exploration feels exciting and new

Tasks:

Create Loop for application of Planet and Sun Texture - rather than creating a method for each planet + the sun and applying the texture manually, using a loop creates efficiency.

Create Orbiting Comets for Saturn's ring - the comets were each created separately for Saturn's ring, to create more realism.

Create Orbit for each planet around the sun - each planet had its own orbital path around the Sun.

Add planets and sun to the background - the Sun and planet objects had to be added to the background sphere.

Apply constant rotation and constant orbit to all elements

Create Background (Stars)

Navigation on Solar System - As a user I want different ways to navigate through the solar system such as: using my mouse, keyboard and sliders on screen so there is more freedom.

Tasks:

Create zoom, rotation, and height sliders (Jslider)

Add sliders to a JPanel on the bottom of the Canvas3d - adding sliders to the bottom helps keep the main view free for the user (while also making sliders accessible)

Create MouseListeners for zoom, rotation and height

Create KeyListeners (WASD, W for moving up, A for moving left, S for moving down and D for moving right)

Create method to update camera/view based on MouseListener, sliders, and KeyListener -

as the Listeners would change the zoom/rotation/height, it was important that the user view matched with it, so a method had to be created to do that.

Add KeyListeners and MouseListeners to the Solar System Canvas3d

Basic Farm Layout - As a user I want a basic farm layout, that I will be transported to through the black hole so I can explore multiple environments

Tasks:

Create Terrain for the farm

Create textures for the farm (TerrainTextures)

Add a path to the farm - having a dirt path on the farm gives the user some direction of where to go

Add a camera that will move as the user requests - this is a Camera object

Add tree objects to the farm, with textures applied - the trees will be repeated across the farm land

Create green grass

Add a blue sky

Add navigation to the farm - navigation will work with the camera

SpaceCraft blender - As a user I want to be within a realistic spacecraft for my journey, to enhance the experience of being in space

Tasks:

Render steering wheel

Render lever

Render start button

Render Stop Button

Render transparent glass window for spacecraft

Add all elements together so they move as one - this is creating a parent child relationship between the elements

Add spacecraft as an image to the screen (keep windows transparent to allow user to click on things through them)

BlackHole tunnel for environment change - As a user I want to travel through a light tunnel that makes it look like I'm travelling at light speed, so the environment change from Solar System to Farm doesn't feel sudden and too unexpected

Tasks:

Make a blender tunnel using Blender 4.3.2 (Cylinder object)

Add emission/stars to the tunnel to enhance the effect of light-speed travel - this creates visual effect to keep user engaged

Research about possible ways to create tunnel - there were many ways to create the transition tunnel, it couldve been done using Java3D OR Blender so research was important

Create animation with tunnel (in Blender) so it moves on its own

Download tunnel animation as collection of PNGs - PNGs are easier to manage than the animation video

Goop Effect - As a user I want random alien attack goop effects on screen for adrenaline and unpredictability

Tasks:

Create Goop PNG - the goop effect was created as a PNG which would appear on a JLabel

Apply it to screen at intervals - the effect was intended to appear on the spacecraft "windshield" at random intervals, but the randomized part did not work as intended.

BlackHole portal - As a user I want to be able to click a black hole to begin my travel through universes for more interaction

Tasks:

Make primitive sphere shape with constant rotation and add user data to make it clickable

Make appearance of black hole: no light, pure opaque black colour

Make sphere appear in random locations of JFrame so the user has to search around for it

Make sphere clickable using pickray and picktools

Attach light tunnel images in a loop to the JLabel once blackhole clicked (so when its clicked the loop begins, and it looks animated)

Add sound to black hole (using SoundJOAL)

Add morphing to the black hole (give it a few different sphere sizes)

Movement of Lever and Steering wheel - As a user I want to be able to move the steering wheel and lever for more realism

Tasks:

Code movement of lever

Code movement of steering wheel

Attach movements to the spacecraft itself - the movement of each piece would have to be attached to the spacecraft so everything moved in unison

Farm Scene - As a user I want the farm to look unrealistic and cartoonish so it contrasts with the solar system and feels otherworldly

Tasks:

Draw cartoonish farm sky

Draw cartoonish farm scene grass

Render farm scene trees

Render farm scene hay

Draw farm scene path

Render a tractor

Render a windmill

Announcer Voice - As a User I want an announcer in the spacecraft so I can be given context and guided through the journey

Tasks:

Create script for announcer voice

Use VoiceMod to generate audios for script (use members voice to record sentences then change the voice)

Add the announcements to the class

Create announcement jingle

Farm Animals - As a user I want cartoon animals (that don't exist on Earth) on the farm for excitement

Tasks:

Get familiar with Blender grease pencil - grease pencil is used for illustrations in Blender

2D render Moobaa (cow + sheep) in blender

2D render Dunny (duck + bunny) in blender

2D render Qubee (quail + bee) in blender

2D render Chonkey (chicken + donkey) in blender

Draw Moobaa in procreate

Draw Dunny in procreate

Draw Qubee in procreate

Draw Chonkey in procreate

Connect Solar System and Farm - As a user I want a seamless connection between the farm and solar system so the journey is smooth and believable

Tasks:

Add black hole to solar system scene

Connect one end of black hole to the farm

Figure out relationship between classes

Research relationship between LWJGL and Java3D - this research is important to ensure that theres compatibility between the environments

Appendix B: Signatures for Contribution Table

Each person signing this appendix has given their approval for the contribution table attached as the Microsoft Excel file.

Kulsum Khan:

A stylized, cursive handwritten signature in black ink, featuring a large initial 'K' and a long, sweeping horizontal stroke at the end.

Makhsuma Khamzaliyeva:

A cursive handwritten signature in black ink, with a large, circular initial 'M' and a long, sweeping horizontal stroke at the end.

Hamnah Riaz Riaz:

A cursive handwritten signature in black ink, with the name 'Hamnah Riaz' written in a clear, flowing script.

Simbarashe Mamvura:

A cursive handwritten signature in black ink, with a large, stylized initial 'S' and a long, sweeping horizontal stroke at the end.

Mahnoz Akhtari:

A handwritten signature in black ink, appearing to read 'Mahnoz', with a long, sweeping horizontal stroke extending to the right.

Appendix C: Signatures for Participation Table

Each person signing this appendix has given their approval for the participation table attached as the Microsoft Excel file.

Kulsum Khan:

A stylized, cursive handwritten signature in black ink, featuring a large initial 'K' and a long, sweeping horizontal stroke at the end.

Makhsuma Khamzaliyeva:

A cursive handwritten signature in black ink, with a large, circular initial 'M' and a long, sweeping horizontal stroke at the end.

Hamnah Riaz Riaz:

A cursive handwritten signature in black ink, with the name 'Hamnah Riaz' written in a clear, flowing script.

Simbarashe Mamvura:

A cursive handwritten signature in black ink, with a large, stylized initial 'S' and a long, sweeping horizontal stroke at the end.

Mahnoz Akhtari:

A handwritten signature in black ink, appearing to read 'Mahnoz', with a long, sweeping horizontal stroke extending to the right.